

ターン制戦闘 R P G の処理



クラス間の階層構造（集約クラスを上位にした整理）

「複数オブジェクトを束ねるクラス」を上位に置いて、`main.py` / `app\_pygame.py` / `adapters/flask\_app.py` で共通に使われる戦闘周辺の階層を示します。

1. ドメイン中核の階層

```
RuntimeState          # 全マスタ・save を保持
└─ (入力として渡される)
BattleSession         # 1戦闘セッションの集約ルート
├─ state: RuntimeState
├─ level_table: LevelTable
├─ party_members: list[PartyMemberRuntime]
│   └─ PartyMemberRuntime
│       ├── job: Job
│       ├── base: BaseCharacter
│       ├── stats: FinalCharacterStats
│       ├── state: BattleActorState
│       └─ equipment: EquipmentSet
├─ enemies: list[EnemyRuntime]
│   └─ EnemyRuntime
│       ├── stats: FinalEnemyStats
│       └─ state: BattleActorState
├─ party_magic_info: Any
├─ party_magic_lists: Any
└─ spells_expanded: dict
```

最上位の集約ルート

: `RuntimeState`
(全体データ)

戦闘ユースケースの
集約ルート

`BattleSession`

4. エントリーポイント別の「上位集約」

- `main.py`: `RuntimeState` を生成し、`BattleSession` を構築してターン進行を管理。
- `app\_pygame.py`: `GameFlowController` が内部に `RuntimeState` を持ち、戦闘単位では `BattleContext` を生成して使用。
- `adapters/flask\_app.py`: `create\_app()` 内で `BattleSession` を保持し、各HTTPルートがそれを操作。

2. 戦闘 1 ラウンド実行時の階層

```
BattleSession
├─ execute_round / execute_round_dto
│   └─ RoundExecutionResult
│       ├── logs: list[str]
│       ├── round_result: SideTurnResult
│       └─ event: list[BattleEvent]
```

3. UIアダプタ側での集約クラス（pygame）

```
GameFlowController    # pygame全体進行の集約
├─ state: RuntimeState
├─ level_table: LevelTable
├─ build_battle_context(...)
│   └─ BattleContext    # 戦闘UI-ドメイン橋渡しの集約
│       ├── party_members: list[PartyMemberRuntime]
│       ├── enemies: list[EnemyRuntime]
│       ├── spells_expanded: dict
│       ├── se: BattleSE
│       ├── bgm: BattleBGM
│       └─ ロジック関数参照群
```

(BattleFlow 内)

```
├─ BattleAppContext    # 入力UI専用の集約
│   ├── party_members / enemies
│   ├── コマンド生成関数群
│   └─ on_committed(), all_actions_committed()
```

UIアダプタ側の集約ルート

- pygame:
`GameFlowController` →
`BattleContext` /
`BattleAppContext`

- Flask: `create\_app()` が
保持する `battle\_session`
(= `BattleSession`)

main.py / app_pygame.py / flask_app.py に共通する主要要素の層分類

UI層（共通）	- <code>`ask_actions_for_party`</code> / pygame入力ハンドラ / Flask JSON受信など、UIの具体実装は異なるため「共通関数」は基本ありません。 - ただし3者とも、最終的には同じ Application 層（<code>`BattleSession`</code>、<code>`build_battle_session`</code>、<code>`execute_round*`</code>）に接続します。
Application層（共通）	- <code>`combat.runtime_state.init_runtime_state`</code> - 各エントリーポイント起動時の初期状態生成 - <code>`combat.usecases.BattleSession`</code> - 各UIが共有する戦闘ユースケース境界オブジェクト - <code>`combat.usecases.build_battle_session`</code> - <code>`RuntimeState`</code> と敵名から戦闘セッションを構築 - （補足）<code>`execute_round`</code> / <code>`execute_round_dto`</code> 呼び出しの順序制御 - <code>`main.py`</code> は直接 <code>`execute_round`</code> - <code>`flask_app.py`</code> は <code>`execute_round_dto`</code> - <code>`app_pygame.py`</code> 側は <code>`BattleController`</code> 経由で <code>`execute_round`</code>
ロジック層（共通）	- <code>`combat.char_build.build_party_members_from_save`</code> - セーブデータから戦闘用パーティを構築 - <code>`combat.enemy_selection`</code> - <code>`build_location_index`</code>、<code>`pick_enemy_names`</code>（敵選出系） - <code>`combat.usecases.execute_round`</code> - 1ラウンド戦闘解決の中核 - Flask は <code>`execute_round_dto`</code> 経由だが内部で同ラウンド実行ロジックに合流 - <code>`combat.progression.apply_victory_rewards`</code> - 勝利報酬適用
データ層（共通）	- <code>`combat.runtime_state.RuntimeState`</code> - 全エントリーポイントで共有される実行時データのコンテナ - <code>`combat.models`</code> - <code>`PartyMemberRuntime`</code>、<code>`EnemyRuntime`</code>、<code>`PlannedAction`</code>、<code>`SideTurnResult`</code> など - 戦闘セッションの入出力を構成する共通データモデル - <code>`system.exp_system.LevelTable`</code> - 経験値テーブルのデータ表現（<code>`build_battle_session`</code> 内でも使用）

どこまでを「共通」とみなすか

- 本書では「同一の関数/クラスを直接または明確な中継経路で再利用している」ものを共通と定義。
- 例えば ``apply_victory_rewards`` は、CLI/pygame/Flask のいずれの戦闘完了フローでも使用されるため共通扱い。

main.py 起点の主要コンポーネント層分類

UI層 (表示・入出力 (CLI)を担当)	- <code>main.py</code> - <code>choose_location_console()</code> の <code>print()</code>/<code>input()</code> によるUI - <code>main()</code> での <code>print_*</code> 系デバッグ表示 - <code>combat.debug_utils</code> - <code>print_party_debug_summary</code>, <code>print_inventory</code>, <code>print_enemies_status_compact</code>, <code>print_round_header_and_state</code>, <code>print_planned_actions</code>, <code>print_logs</code>, <code>print_end_reason</code>	- <code>combat.input_ui</code> - <code>ask_actions_for_party(..., input_func=input, output_func=print)</code> のCLI入出力
Application層 (ユースケース 呼び出し順序・ フェーズ進行・ トランザクシ ョン境界を担当)	- <code>main.py</code> - <code>main()</code> - <code>init_runtime_state()</code> で状態初期化 - 場所選択に必要な事前計算 (平均レベル算出) - <code>build_battle_session()</code> でセッション構築 - ターンループ内で <code>ask_actions_for_party()</code> → <code>execute_round()</code> を反復 - 勝利時に <code>apply_victory_rewards()</code> → セーブ処理 - <code>choose_location_console()</code> - ロケーション候補一覧の提示と選択フロー管理	- <code>combat.save_prompt</code> - <code>prompt_save_progress_and_write</code>, <code>restore_backup_by_choice</code> - 勝利後の保存可否確認/バックアップ復元フローを提供
ロジック層 (戦闘ルール、 算出、判定等の ドメインロジ ックを担当)	- <code>combat.enemy_selection</code> - <code>build_location_index</code>, <code>calc_party_avg_level</code>, <code>danger_label</code>, <code>pick_enemy_names</code> - <code>combat.char_build</code> - <code>build_party_members_from_save</code> - <code>combat.usecases</code> - <code>build_battle_session</code>, <code>execute_round</code>	- <code>combat.input_ui</code> - <code>ask_actions_for_party</code> (入力自体はUI由来だが、コマンド候補や対象判定など戦闘ルールに密接) - <code>combat.progression</code> - <code>apply_victory_rewards</code> - <code>combat.debug_utils</code> - <code>check_battle_end_before_round</code> (終了条件判定)
データ層 (永続 データの読込・ 保持、構造体定 義を担当)	- <code>combat.runtime_state</code> - <code>RuntimeState</code> (モンスター/武器/防具/魔法/アイテム/ジョブ/saveを保持) - <code>init_runtime_state()</code> (JSON群をロードして <code>RuntimeState</code> を構築) - <code>assets.data.data_loader</code> - <code>load_monsters</code>, <code>load_weapons</code>, <code>load_armors</code>, <code>load_spells</code>, <code>load_items</code>, <code>load_jobs</code>, <code>load_savedata</code>	- <code>combat.models</code> - 主要データクラス (<code>PartyMemberRuntime</code>, <code>EnemyRuntime</code>, <code>PlannedAction</code>, <code>SideTurnResult</code> など) - <code>system.exp_system</code> - <code>LevelTable</code>, <code>LevelStatus</code> (レベル/経験値テーブルの読込・状態表現)

補足 (境界の見方) - `combat.input_ui` は **UI (入出力) とロジック (行動入力 of 妥当性) ** が混ざりやすい境界モジュール。このため「UI層にもロジック層にも接続する境界」として扱うと整理しやすいです。- `main.py` はフレームワーク非依存なオーケストレータで、設計上は Application層の中心に位置づきます。

app_pygame.py 起点の主要コンポーネント層分類

UI層 (描画・入力・pygame依存)	- <code>`ui_pygame.battle_flow`</code> - <code>`run_one_battle`</code> (pygameイベント処理、描画呼び出し) - <code>`ui_pygame.input_handler`</code> - <code>`handle_keydown`</code> (入力モード別ディスパッチ) - <code>`ui_pygame.enemy_flow`</code> - <code>`choose_location_group_pygame`</code>, <code>`choose_location_floor_pygame`</code> - <code>`ui_pygame.render.*`</code> - <code>`draw_header`</code>, <code>`draw_enemy_panel`</code>, <code>`draw_party_panel`</code>, <code>`draw_log_panel`</code>, <code>`draw_command_panel`</code> 等 - <code>`ui_pygame.victory_flow`</code>, <code>`ui_pygame.gameover_flow`</code> - 勝利画面/ゲームオーバー画面の表示
Application層 (ユースケース実行・フロー制御)	- <code>`app_pygame.py`</code> - エントリーポイント (<code>`BattleAppConfig`</code> 生成→<code>`run_pygame_app`</code> 呼び出し) - <code>`ui_pygame.app`</code> - <code>`run_pygame_app`</code> (pygame初期化/終了、ゲームループ起動) - <code>`ui_pygame.game_loop`</code> - <code>`run_game_loop`</code> (<code>`GameFlowController`</code> 起動) - <code>`ui_pygame.game_flow_controller`</code> - <code>`GameFlowController`</code> - <code>`run`</code>, <code>`prepare_battle`</code>, <code>`build_battle_context`</code>, <code>`handle_victory`</code> - <code>`ui_pygame.game_flow_service`</code> - <code>`prepare_battle_resources`</code>, <code>`select_enemy_names_from_groups`</code> - <code>`ui_pygame.battle_context`</code> - <code>`BattleContext`</code> (戦闘ユースケースへ渡す依存束) - <code>`ui_pygame.controller`</code> - <code>`BattleController.update`</code>, <code>`_resolve_one_round`</code> (UI状態とドメイン実行の橋渡し)
ロジック層 (ドメイン/業務ルール)	- <code>`combat.usecases`</code> - <code>`BattleSession`</code>, <code>`build_battle_session`</code>, <code>`execute_round`</code> - <code>`ui_pygame.logic`</code> - <code>`reset_target_flags`</code>, <code>`make_planned_action`</code>, <code>`get_job_commands`</code>, <code>`build_item_candidates_for_battle`</code> - <code>`combat`</code> 配下の計算/判定関数 - <code>`compute_character_final_stats`</code>, <code>`normalize_battle_command`</code>, <code>`is_out_of_battle`</code>, <code>`apply_victory_rewards`</code>, <code>`expand_spells_for_summons`</code> など
データ層	- <code>`assets.data.data_loader`</code> - <code>`_load_named_index`</code>, <code>`load_monsters`</code>, <code>`load_weapons`</code>, <code>`load_armors`</code>, <code>`load_spells`</code>, <code>`load_items`</code>, <code>`load_jobs`</code>, <code>`load_savedata`</code>, <code>`load_explicit_groups`</code> - <code>`combat.runtime_state`</code> - <code>`RuntimeState`</code>, <code>`init_runtime_state`</code> - <code>`combat.models`</code> - 主要データクラス (<code>`PartyMemberRuntime`</code>, <code>`EnemyRuntime`</code>, <code>`PlannedAction`</code>, <code>`BattleActorState`</code>, <code>`SideTurnResult`</code> など) - <code>`system.exp_system`</code> - <code>`LevelTable`</code>, <code>`LevelStatus`</code> - <code>`system.cp_system`</code> - <code>`CPStatus`</code>, <code>`load_job_attribution`</code> (CSV読込)

- ``BattleAppContext`` は UI操作コンテキスト (``on_committed``, ``all_actions_committed`` を持つ) で、**UI層寄りのApplication層境界**として機能。
- ``BattleContext`` は戦闘ユースケースに必要な依存束で、**Application層の境界オブジェクト**。

flask_app.py 起点の主要コンポーネント層分類

UI層 (HTTP/HTML/JS ONという入出カ インターフェースを担 う層)	- adapters/flask_app.py の Flask ルート - 画面系: <code>`GET /`, `GET /menu`, `GET /menu/item`, `GET /menu/magic`, `GET /menu/equip`, `GET /menu/status`, `GET /menu/job`</code> - 戦闘API: <code>`POST /battle/round`</code> - メニューAPI: <code>`GET /menu/state`, `POST /menu/use-item`, `POST /menu/cast-magic`, `POST /menu/magic/learn`, `POST /menu/magic/remove`, `POST /menu/magic/swap`, `POST /menu/magic/use`, `POST /menu/change-equipment`, `POST /menu/unequip-all`, `POST /menu/change-job`, `POST /menu/reorder`, `POST /menu/toggle-row`, `POST /menu/save`</code> - 静的アセット: <code>`/assets/enemy-sprites/*`, `/assets/portraits/*`, `/assets/ui/location-group-bg/*`, `/assets/status-icons/*`</code> - templates/*.html - <code>`top.html`, `menu.html`, `menu_item.html`, `menu_magic.html`, `menu_equip.html`, `menu_status.html`, `menu_job.html`</code> - adapters/flask_error_handlers.py - <code>`jsonify`</code> による HTTP エラー応答フォーマット	
Application層 (ユースケース呼び 出し順序・セッシ ョン管理・フロー制 御を担う層)	- adapters/flask_app.py - <code>`create_app()`</code> (依存初期化、ルート登録、<code>`battle_session`</code> 管理) - <code>`_build_default_session()`, `_refresh_session_party()`</code> (セッション再構築/同期) - <code>`_normalize_planned_actions_length()`, `_require_int()`, `_require_str()`</code> (入力のアプリ境界検証) - <code>`_build_*_snapshot()`, `_build_menu_state_payload()`</code> (テンプレート/API向け ViewModel 組み立て) - <code>`run_dev_server()`</code> (実行設定の起動ラッパ)	- adapters/flask_error_handlers.py - <code>`register_flask_error_handlers(app)`</code> (アプリ全体の例外ハン ドリング方針を登録)
ロジック層 (戦闘ルール・判 定・計算・ドメイン エラー変換などを担 う層)	- <code>`combat.usecases`</code> - <code>`build_battle_session`, `execute_round_dto`</code> - <code>`combat.enemy_selection`</code> - <code>`build_location_index`, `build_groups`, `pick_enemy_names`</code> - <code>`combat.char_build`</code> - <code>`build_party_members_from_save`, `apply_job_equipments_restrictions`, `compute_character_final_stats`</code> - <code>`combat.progression`</code> - <code>`apply_victory_rewards`</code> - <code>`combat.magic_menu`, `combat.magic_damage`</code> - <code>`build_party_magic_info`, `build_party_magic_lists`, `healing_spell_kind`</code>	- <code>`combat.input_ui`</code> - <code>`normalize_battle_command`</code> - <code>`system.cp_system`</code> - <code>`compute_job_change_cp_cost`, `load_job_attribution`</code> - adapters/flask_menu_actions.py - <code>`make_cast_field_magic_fn`, `make_use_field_item_fn`</code> (フィールド魔法/アイテムの適用 ロジック) - adapters/flask_error_handlers.py - <code>`map_domain_error_to_http`, `map_unexpected_error_to_http`</code>
データ層 (永続データ・構造 体・DTOなど「デー タ保持/変換」を中 心に担う層)	- <code>`combat.runtime_state`</code> - <code>`RuntimeState`, `init_runtime_state()`</code> (JSONマスタとセーブデータを読み込み) - <code>`assets.data.data_loader`</code> - <code>`load_explicit_groups`, `save_savedata`</code> - <code>`combat.models`</code> - <code>`EquipmentSet`</code> ほか戦闘データクラス (<code>`PartyMemberRuntime`, `EnemyRuntime`, `PlannedAction`</code> など)	- combat.dto - <code>`ExecuteRoundInputDTO`, `parse_execute_round_input_dict`, `to_json_ready_dict`</code> - <code>`system.exp_system`</code> - LevelTable, <code>`LevelStatus`</code>

補足 (境界の見方)

- **adapters/flask_app.py** には、純粋な UI (ルーティング/レスポンス) だけでなく、 画面表示用スナップショット生成のような Application 的責務も含まれます。

- **adapters/flask_menu_actions.py** は adapter 配下にありますが、 実際には HP/MP/状態異常・在庫更新などの業務ロジックを持つため、「Flask から呼び出されるロジック層の境界モジュール」
として扱うと整理しやすいです。